

Pitch to Postgres: Designing and Querying a Relational Database for FIFA World Cup Data

Viva notes

Group: Devanshi Dudhatra (202618027), Pujita Sunapu (202618003), Rahul Saha (202618037). DS604 Introduction to Data Management. **Data:** The Fjelstul World Cup Database by Joshua C. Fjelstul, <https://github.com/jfjelstul/worldcup>, CC-BY-SA 4.0.

Short, speakable answers. The numbers are from our own database and were checked.

1. Why each table has its primary key

General rule. We use the dataset's own identifiers (natural keys such as `T-03` or `M-1930-01`). In Milestone 0 we checked that each is unique and never missing. They are stable, and they let child tables be appended with `to_sql` without renumbering foreign keys. `key_id` in the CSVs is only a row number, so it is not a key.

Table	Primary key	Why this key
confederations	confederation_id	One id per confederation; name and code are also UNIQUE (alternate keys).
positions	position_code	Our lookup: the code (GK, CB, ...) identifies the position; the name is UNIQUE.
awards	award_id	One id per award type; award_name is UNIQUE.
stadiums	stadium_id	The name is not unique (two "Olympiastadion"s), so the id is needed; (name, city) is UNIQUE.
teams	team_id	team_code cannot be the key: Germany and West Germany share "DEU". team_name is UNIQUE.
players	player_id	Names repeat (6,440 family names for 7,907 players); the id is the only reliable identifier.
managers	manager_id	Same reason as players.
referees	referee_id	Same reason as players.
tournaments	tournament_id	One id per edition; year and name are also UNIQUE, and a CHECK ties id to year (<code>WC-1930</code> = 1930).
tournament_stages	(tournament_id, stage_number)	A stage only exists within a tournament (weak entity); the number gives the order. (tournament_id, stage_name) is also UNIQUE.
groups	(tournament_id, stage_number, group_name)	Group names repeat across stages (1950 has "Group 1" in two stages), so the stage must be in the key.
qualified_teams	(tournament_id, team_id)	A team takes part in a tournament once; this is the bridge between tournaments and teams.
host_countries	(tournament_id, team_id)	A tournament can have two hosts (2002); each host appears once.
tournament_standings	(tournament_id, position)	Each of positions 1-4 is filled once; (tournament_id, team_id) is also UNIQUE.
group_standings	(tournament_id, stage_number, group_name, position)	One team per table position in a group; (tournament_id, stage_number, team_id) is also UNIQUE.

Table	Primary key	Why this key
squads	(tournament_id, team_id, player_id)	A player is listed once in a squad; UNIQUE (tournament_id, player_id) adds "one squad per tournament".
manager_appointments	(tournament_id, team_id, manager_id)	Joint managers exist (12 cases), so manager_id must be part of the key.
referee_appointments	(tournament_id, referee_id)	A referee is appointed to a tournament once.
matches	match_id	One id per match; replays are separate matches with their own ids.
team_appearances	(match_id, team_id)	Exactly two rows per match, one per team.
referee_appearances	match_id	One referee per match, so the match alone identifies the row (1:1 with matches).
manager_appearances	(match_id, team_id, manager_id)	42 team-matches had two managers.
player_appearances	(match_id, player_id)	A player appears at most once in a match.
goals	goal_id	Each goal is an individual event; nothing else identifies it (a player can score twice in one minute).
bookings	booking_id	Each card is an event.
substitutions	substitution_id	Each going-off or coming-on row is an event.
penalty_kicks	penalty_kick_id	Each shoot-out kick is an event.
award_winners	(tournament_id, award_id, player_id)	Ties produce several winners of the same award (six for the 1962 Golden Boot).

2. Join logic of each query in plain words

#	Query	Join logic in plain words
Q01	Most successful teams	Take each top-four finish and attach the team's name; count positions 1, 1-2 and 1-4 per team.
Q02	All-time team records	Take every team-in-a-match row, mark it won/drawn/lost from the goals, attach the team name, and add up per team (only teams with 10+ matches).
Q03	Host advantage	Take every team-in-a-match row, attach the tournament year, and left join to the hosts. If a host row is found the team was a host, otherwise not. Compare the two groups.
Q04	Confederations by decade	Team-in-a-match → tournament (for the decade) → team → confederation; average the win flag per confederation and decade.
Q05	Never past the opening round	For each qualified team, keep it only if no match of theirs is in a stage numbered above 1 (NOT EXISTS joining their appearances to the matches).
Q06	Top four but no title	Teams with a top-four finish EXCEPT teams with a first place; join the remaining teams back to their finishes and years.
Q07	Head-to-head	Self-join: pair each team's row in a match with the <i>other</i> team's row in the same match, keep the pair whose names match the two chosen teams, then attach match, tournament and stage.
Q08	Cumulative titles	Champions (position 1) joined to their tournament and name; a running count per team over the years; a subquery finds who leads after each year.
Q09	Comebacks	Number the goals in each match by minute to find the first scorer; join every team-in-a-match row to that first goal where the first scorer was the <i>other</i> team; count wins.
Q10	Top scorers	Goals (not own goals) joined to the scorer and the scorer's team; count per player, rank, keep ranks 1-10 (ties kept).

#	Query	Join logic in plain words
Q11	Top scorer vs Golden Boot	Count goals per player per tournament and rank within each tournament; separately, award winners joined to awards (Golden Boot only). For each tournament, compare the two sets in both directions with EXCEPT.
Q12	Hat-tricks	Each goal joined to the scorer, the scorer's team row in that match (which gives the opponent and score), the stage and the year; keep player-match groups with 3+ goals.
Q13	Substitute goals	Each goal joined to the scorer's appearance in the same match (on match and player); if that appearance was not as a starter, the goal came from a substitute.
Q14	Squad age	Squad entries joined to the player (birth date) and the tournament (start date); age = start date - birth date; averaged per position and year.
Q15	Most appearances per nation	Count appearances per (team, player); number players within each team by appearances; keep number 1; attach team and player names.
Q16	Players for two teams	Squad entries joined to player, team and tournament; group by player and keep those with more than one distinct team.
Q17	Goals per match + LAG	Matches joined to their tournament; total goals and matches per year; LAG looks at the previous year's row.
Q18	Biggest margins / most goals	Matches joined to both teams (the teams table twice, as home and away), the year and the stage; ranked two ways; keep the top two of either ranking.
Q19	When goals are scored	No join: each goal is put in a time band by its minute, then counted.
Q20	Penalties and own goals	Goals joined to the tournament year; count penalty and own goals per year; compare with the average of earlier years.
Q21	Shoot-outs	Shoot-out results per team (from team rows with penalty scores) joined to kick statistics per team (from the kicks table) and the team name.
Q22	Strictest referees	Each referee-match joined to the year and left joined to the cards of that match, so card-free matches count as 0; then averaged per referee (5+ matches).
Q23	Card trend	Matches since 1970 left joined to their cards; count per year; moving average over three tournaments.
Q24	Cards by stage	Matches joined to stage and year, left joined to cards; classify the stage as group or knockout; average per stage.
Q25	Most-carded teams	Cards joined to the team; a correlated subquery counts each team's matches since 1970 to get a rate.
Q26	Busiest stadiums	Matches joined to stadium, tournament and stage; count matches, tournaments and finals per stadium; rank.
Q27	Managers	Each manager-in-a-match joined to the team's row in that match on both match and team (composite key) to get the result, plus the manager and team names.
Q28	Foreign vs home managers	Appointments joined to manager and team to flag nationality; one label per team-tournament; left join to standings for top-four finishes; a subquery counts matches.
Q29	Golden Ball	Award winners joined to award, player, team and year; left join to standings, because many winners' teams did not finish in the top four.
Q30	Integrity check	Every team-in-a-match row left joined to the goal count for that team and match (missing = 0); compare the two numbers with EXCEPT; NOT EXISTS finds goals pointing at a team that was not in the match.

Views (Milestone 5): each view is one of the queries above saved under a name. `v_match_results` joins matches to stage, stadium and both teams; `v_head_to_head` groups team rows by (team, opponent); the others follow Q01/Q02, Q03, Q10, Q22 and the tournament-level parts of Q11/Q17/Q23.

3. Likely viva questions, with short answers

Design and normalisation

Q: What normal form is your schema in? Third normal form. Every non-key column depends on the key, the whole key, and nothing but the key. We removed copies of names and codes (2NF/3NF violations), the comma-separated `list_tournaments` (1NF), and values derived from the same row, such as margins, result labels and points. One deliberate exception is `tournament_id` in the event tables (see below).

Q: Why keep `tournament_id` in `goals` if it depends on `match_id`? It lets us write the foreign key `(tournament_id, player_team_id, player_id) -> squads`, which guarantees every scorer was in that team's squad for that tournament. The redundancy cannot cause inconsistency, because a second foreign key `(match_id, tournament_id) -> matches(match_id, tournament_id)` forces it to agree with the match.

Q: Natural or surrogate keys, and why? Natural keys, meaning the dataset's ids. They are unique and stable (verified), they carry meaning (`WC-1930`, `M-1930-01`), and with append-only loading, child tables can be loaded as they are. Surrogate keys would require mapping every foreign key during loading.

Q: Why are there composite keys? In bridge tables (`squads`, `qualified_teams`, ...) and weak entities (`tournament_stages`, `groups`), one row is identified by the combination of parents. For example, a squad entry is (tournament, team, player).

Q: How did you handle teams that no longer exist? West Germany, Soviet Union, Yugoslavia and others stay separate teams with their own ids, as in the source. That is why `team_code` is not unique (Germany and West Germany share DEU). We did not add a lineage table, because merging teams is outside knowledge. Queries say when this matters (e.g. Matthäus' appearances are split 16 + 9).

Q: How do you know you lost no information when dropping columns? We ran 33 checks proving every dropped column can be recomputed exactly from what we keep. For example, `result` from the scores and shoot-out scores, and `points` with 2 points per win to 1990 and 3 from 1994. The views rebuild the readable ones.

Q: Why do you store NULL instead of "not available"? NULL is SQL's marker for unknown. Functions such as AVG and COUNT(column) skip it correctly, CHECKs and types work, and a text marker in a DATE column is impossible anyway.

Q: What does the deferred foreign key do? `team_appearances.(match_id, opponent_id)` must point at the opponent's own row in the same match. Because both rows of a match are inserted together, the check is DEFERRABLE INITIALLY DEFERRED: it runs at COMMIT, not after each row.

Q: Which rules could you not enforce with constraints? Rules that compare across tables: a match date within its tournament's dates, a group name present only for group stages, and goals adding up to the score. These need triggers, so we check them with queries instead (load notebook checks, Q30), and all pass.

Loading

Q: Why must the DDL run before `to_sql(if_exists="append")`? Append inserts rows into an existing table. If the table did not exist, pandas would create one with its own guessed types and no keys or constraints. Running our DDL first means the data must fit our types, primary keys, foreign keys and CHECKs, and the load fails loudly if it does not.

Q: What happens if you run the load twice? The primary keys reject duplicate rows. The notebook also checks first and refuses to load into tables that already contain data, and the whole load is one transaction, so a failure leaves the database empty.

Q: Why load in a particular order? Foreign keys require the parent row to exist first: tournaments before stages, before groups, before matches, before goals, and so on.

Q: What cleaning did you do? Seven explicit steps: 1. Text markers → NULL (11,891 cells). 2. Shirt number 0 → NULL. 3. Look up `stage_number` for matches. 4. Align 18 group names. 5. NULL shoot-out scores when there was no shoot-out. 6. Keep only the normalised columns. 7. Convert types.

Q: What surprised you in the data? - The source records shoot-out winners as "wins", so the raw flags give Brazil 76 wins instead of FIFA's 73. - Stage and group names differ between tables (1950, 1974-1982); our foreign keys caught the 1982 group names, and an integrity check caught the 1950 final round. - The 1998 joint managers of Saudi Arabia and Tunisia are listed with the same foreign nationality.

Indexes and performance

Q: Why these indexes? Primary keys and UNIQUE constraints are indexed automatically. We indexed the foreign-key columns that queries join or filter on and that are not already the first column of an index, for example `goals(player_id)`, `goals(match_id, team_id)`, `team_appearances(team_id, opponent_id)` and `matches(stadium_id)`. PostgreSQL does not index foreign keys by itself, and these indexes also speed up checks when a parent row is deleted.

Q: Do the indexes matter with only 57,269 rows? Only a little. Everything runs in milliseconds. They follow good practice and would matter on a larger database, and `EXPLAIN` can show them being used.

Queries

Q: WHERE vs HAVING? WHERE filters rows before grouping; HAVING filters groups after aggregation, e.g. `HAVING COUNT(*) >= 10` in Q02.

Q: RANK vs DENSE_RANK vs ROW_NUMBER? - **RANK** gives ties the same rank and leaves gaps (1, 2, 2, 4). - **DENSE_RANK** gives ties the same rank without gaps (1, 2, 2, 3). - **ROW_NUMBER** gives every row a unique number, even ties (used in Q15 to pick exactly one player per team).

Q: What is a correlated subquery? Give an example. A subquery that refers to the outer row and is evaluated per row. Q25 counts each team's matches since 1970 using `ta.team_id = t.team_id` from the outer query.

Q: Why LEFT JOIN in the discipline queries? Matches without any card have no booking rows. An inner join would drop them and inflate the average. LEFT JOIN keeps them and `COUNT(booking_id)` gives 0.

Q: How does the moving average work? `AVG(...) OVER (ORDER BY year ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)` averages each tournament with the two before it (Q23).

Q: Why do you count a shoot-out as a draw? That is FIFA's official convention: the match result is a draw, and the shoot-out only decides who advances. With it, our numbers match FIFA's records (Brazil 109 played, 73 won, 18 drawn, 18 lost).

Q: Did you find any bugs in your own SQL? Yes. `FETCH FIRST 10 ROWS WITH TIES` with a name as tie-breaker dropped tied rows, because rows only count as tied if the whole ORDER BY is equal. Comparing the top-scorer view with Q10 exposed it: Rahn, Cubillas and T. Müller (10 goals) were missing. We now rank and filter on `rank <= 10`.

Q: How did you validate the results? 18 checks against pandas on the raw CSV files (bypassing the database) and against well-known facts: Klose 16 goals, Fontaine 13 in 1958, 2,548 goals, Hungary 10-1 El Salvador, Azteca 19 matches, and Schön 25 matches with 16 wins. 16 match exactly; 2 differ for documented reasons (Matthäus split across two teams; 2010 Golden Boot decided on assists).

Views

Q: View versus table? A view is a stored query: it holds no data and is recomputed each time, so it is always up to date and cannot disagree with the tables. A table stores data and must be maintained. Views give users one shared definition, for example a shoot-out = draw. If a summary were expensive and read very often, a materialized view or summary table would be faster but must be refreshed.

Q: Which views did you create, and why? Seven: - `v_match_results` and `v_tournament_summary` rebuild the readable columns we did not store; - `v_top_scorers` counts per person; - `v_team_performance`, `v_referee_discipline` and `v_host_performance` summarise our main questions; - `v_head_to_head` answers any head-to-head with one `WHERE` clause.

All seven were checked against the queries they came from (8 of 8 checks agree).

Q: Can you update data through these views? No. They contain joins, GROUP BY and window functions, so PostgreSQL treats them as read-only, which is intended: data changes go into the base tables.

Insights and users

Q: How do the insights help real users? - **Federations:** hosts win 63% of matches but less than before 1982 (bidding); teams with home-nation managers won every title (correlation, not cause). - **Coaches:** scoring first is decisive, and only strong teams such as Brazil and West Germany regularly come back; goals cluster in minutes 76-90; substitutes scored up to 19% of goals (2014). - **Referees' bodies:** cards per match peaked in 2006; strict and lenient referees differ widely (8.0 vs 2.0 per match); 2018 (VAR) had a record share of penalty goals. - **Broadcasters and analysts:** consistent all-time tables, head-to-heads and fact sheets from the views.

Q: What can the data not tell you? - Possession, shots, assists, attendance, missed in-match penalties, or exact times. - Anything from 2022, women's tournaments or qualifiers. - Cards and line-ups before 1970. - Historical confederation membership (it records today's).

Practical and ethical

Q: How do you keep credentials safe? They live only in `.env`, which is listed in `.gitignore`. `.env.example` has placeholders. The code builds the connection with `URL.create` and prints only `user@host:port/database`; we checked that the password appears in no notebook or PDF.

Q: How do you move to the course server? Edit `.env` (host 10.100.71.21 and the given credentials), run `src/check_connection.py`, then run notebooks 03, 04 and 05. Nothing else changes.

Q: What does CC-BY-SA require from you? - **Attribution:** credit the author and link the source. - **Changes:** state the modifications we made. - **ShareAlike:** share our derived work under the same licence.

Every document carries this attribution.